A23 B3 Web Technologies for Data-Centric Applications & Services – evaluation

Final Project: Smart City Data Dashboard

Team Size

6 Students

Project Description

Students must design and develop a Smart City Data Dashboard capable of collecting, storing, processing, and visualizing public city data from external APIs.

The application must include:

- external API integration,
- SQL data storage,
- backend REST API,
- authentication system,
- dynamic frontend dashboard,
- charts and analytics.

The project must be developed collaboratively using Git.

All source code must be signed by the student responsible for the implementation.

Example:

```
// Author: John Doe
```

Required Technologies

Frontend

- HTML
- CSS
- Vanilla JavaScript
- Chart.js (https://www.chartjs.org)

Backend

- PHP
- MVC Architecture
- REST API

Database

- MySQL or MariaDB

Version Control (recommended)

- Git

Required External APIs

Weather API

Open-Meteo

Example:

```
https://api.open-meteo.com/v1/forecast?latitude=48.85&longitude=2.35¤t=temperature_2m,relative_humidity_2m,wind_speed_10m
```

Country API

REST Countries

Example:

```
https://restcountries.com/v3.1/name/france
```

Geolocation API

Nominatim OpenStreetMap

Example:

```
https://nominatim.openstreetmap.org/search?q=Paris&format=json
```

Mandatory Features

1. City Management

The application must allow users to:

- search for a city,
- retrieve geographic coordinates,
- store city information in SQL.

2. Weather Data Collection

The application must:

- fetch weather data from Open-Meteo,
- store historical weather records,
- save at least:
    - temperature,
    - humidity,
    - wind speed,
    - timestamp.

3. SQL Database

Students must create a relational SQL database.

Minimum required tables:

- cities
- weather_data

The database must contain:

- primary keys,
- foreign keys,
- normalized relations.

4. Backend REST API

Students must create REST API endpoints.

Minimum required endpoints:

```
POST /api/register

POST /api/login

GET /api/cities

POST /api/cities

GET /api/weather?city_id=1

POST /api/weather/sync

GET /api/stats?city_id=1
```

The backend must follow MVC architecture principles.

5. Dashboard

The frontend must include:

- city selector,
- current weather information,
- dynamic charts,
- statistics.

All data must be retrieved from the backend API using JavaScript.

6. Data Visualization

The application must contain at least:

- 2 charts,
- historical visualization,
- city comparison indicators.

Examples:

- temperature evolution,
- humidity evolution,

- comparison between cities.

7. Analytics

The application must calculate:

- average temperature,
- minimum temperature,
- maximum temperature,
- latest weather record.

Student Roles

Each student is responsible for one complete business feature.

Each feature must include:

- SQL work,
- backend logic,
- API integration,
- frontend integration,
- testing.

Students must collaborate to integrate all modules into a single application.

Student 1: Authentication & Users

Responsible for:

- registration system,
- login/logout,
- password hashing,
- authentication middleware,
- user SQL table,
- protected frontend pages.

Student 2: City Search & Registration

Responsible for:

- city search interface,
- geolocation API integration,
- country API integration,
- city insertion into database,
- city API endpoints,
- frontend integration.

Student 3: Current Weather Module

Responsible for:

- current weather API requests,
- backend weather services,
- SQL insertion,
- weather display cards,
- current weather endpoints,
- frontend integration.

Student 4: Historical Data & Analytics

Responsible for:

- historical weather storage,
- synchronization scripts,
- average/min/max calculations,
- analytics SQL queries,
- analytics endpoints,
- historical charts.

Student 5: Dashboard & Data Visualization

Responsible for:

- dashboard layout,
- navigation,
- chart integration,
- JavaScript dashboard logic,
- API consumption from frontend,
- responsive design.

Student 6: Backend Architecture & Integration

Responsible for:

- MVC structure,
- routing system,
- shared models/services,
- API consistency,
- error handling,
- integration between all modules.

Optional Features

Optional features may improve the final grade.

Examples:

- air quality integration,
- scheduled ingestion script,
- CSV export,
- admin interface,
- Node.js ingestion worker,
- interactive map,
- advanced statistics.

Recommended Database Structure

users

```
id
username
email
password
token
expiration_token
created_at
```

cities

```
id
name
country
latitude
longitude
population
created_at
```

weather_data

```
id
city_id
temperature
humidity
wind_speed
recorded_at
created_at
```

Deliverables

Students must submit:

```
/project-source-code
/database.sql
/README.md
/api-documentation.md
/presentation.pdf
```

README Requirements

The README must contain:

- project description,
- installation steps,
- database configuration,
- API sources,
- team member responsibilities,
- project architecture explanation.

Final Evaluation: Transparent Grading

Category	**Points**	**Evaluation Criteria**
Authentication & Security	15	Login system, password hashing, protected routes
Database Design	15	SQL structure, normalization, relations
External API Integration	20	API requests, parsing, reliability
Data Ingestion & Storage	15	SQL insertion, historical data storage
PHP MVC Backend	15	Routing, controllers, models, API quality
Dashboard & Frontend	10	Dynamic interface, JavaScript integration
Data Visualization & Analytics	5	Charts, statistics, comparisons
Git Collaboration	3	Meaningful commits and teamwork
Documentation & Presentation	2	README quality and final demonstration

Total: 100 Points

Minimum Expected Scope

To validate the project, students must implement:

- city management,
- weather API integration,
- SQL storage,
- authentication
- PHP MVC backend API,
- dynamic dashboard,
- charts and analytics.

Optional features are not mandatory but may improve the final grade.